student@pwr:~$ ./lab4.sh

Wątki i procesy w Pythonie

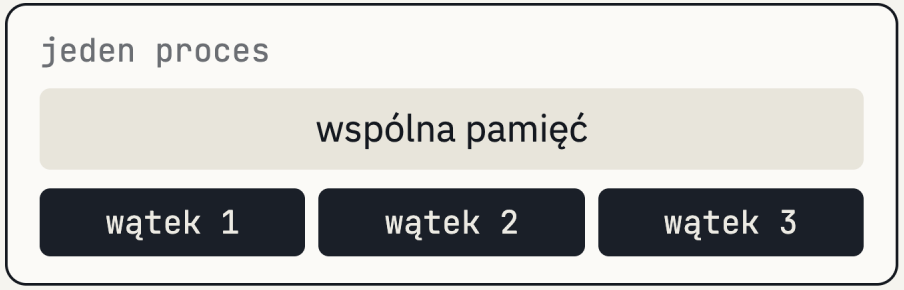
Kiedy wątki naprawdę przyspieszają, co blokuje GIL i jak rzetelnie porównać cztery wersje regresji liniowej.

Przetwarzanie danych masowych · Laboratorium 4 · Politechnika Wrocławska

01 · TEORIA

Wątek a proces

Wątki



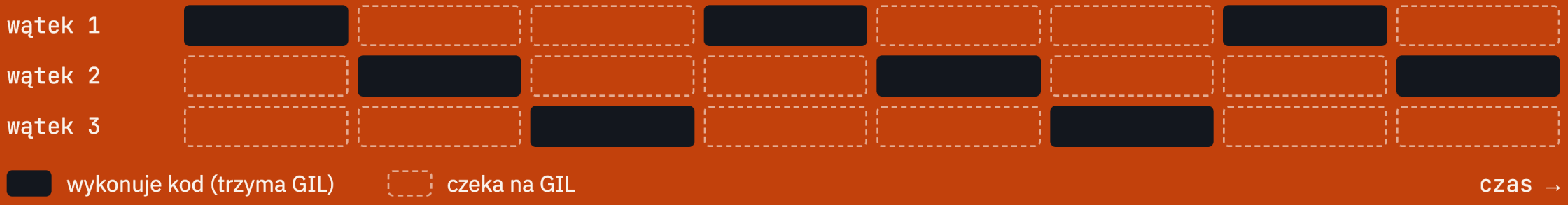
Procesy



dane między procesami: pickle

Tworzenie	≈ 0,08 ms na wątek	fork ≈ 2 ms, spawn ≈ 40 ms (nowy interpreter, ~19 MB)
Pamięć	wspólna dla wszystkich wątków	osobna kopia w każdym procesie
Wymiana danych	bezpośrednio (uwaga na wyścigi)	serializacja pickle i kopiowanie
GIL	jeden na cały proces	każdy proces ma własny

W CPythonie kod Pythona wykonuje naraz tylko jeden wątek.



Wyjątki: czekanie na I/O (sieć, dysk) i kod w C, np. NumPy, zwalniają GIL — tam wątki pomagają.

Od Pythona 3.13 jest też osobny build bez GIL — o nim na następnym slajdzie.

03 · BEZ GIL

Python bez GIL: build free-threaded

3.12	osobny GIL dla każdego subinterpretera (PEP 684)
3.13	eksperymentalny build bez GIL: python3.13t (PEP 703)
3.14	build bez GIL oficjalnie wspierany (PEP 779) i nowy InterpreterPoolExecutor
3.15	nadal osobny, opcjonalny build: python3.15t

```
$ python3.15t -VV
Python 3.15.2 free-threading build …
>>> sys._is_gil_enabled() # False
```

Co zamiast GIL

Blokady wewnątrz dict, list i set, inne liczenie referencji, obiekty nieśmiertelne.

Koszt

Jeden wątek wolniejszy o 1–8% (u nas 8%), więcej pamięci na każdy obiekt.

Rozszerzenia w C

Niegotowy moduł włącza GIL z ostrzeżeniem. NumPy 2.5 ma już wheele dla 3.15t.

Docker

Nie ma obrazów python:*t. Instalujemy przez uv.

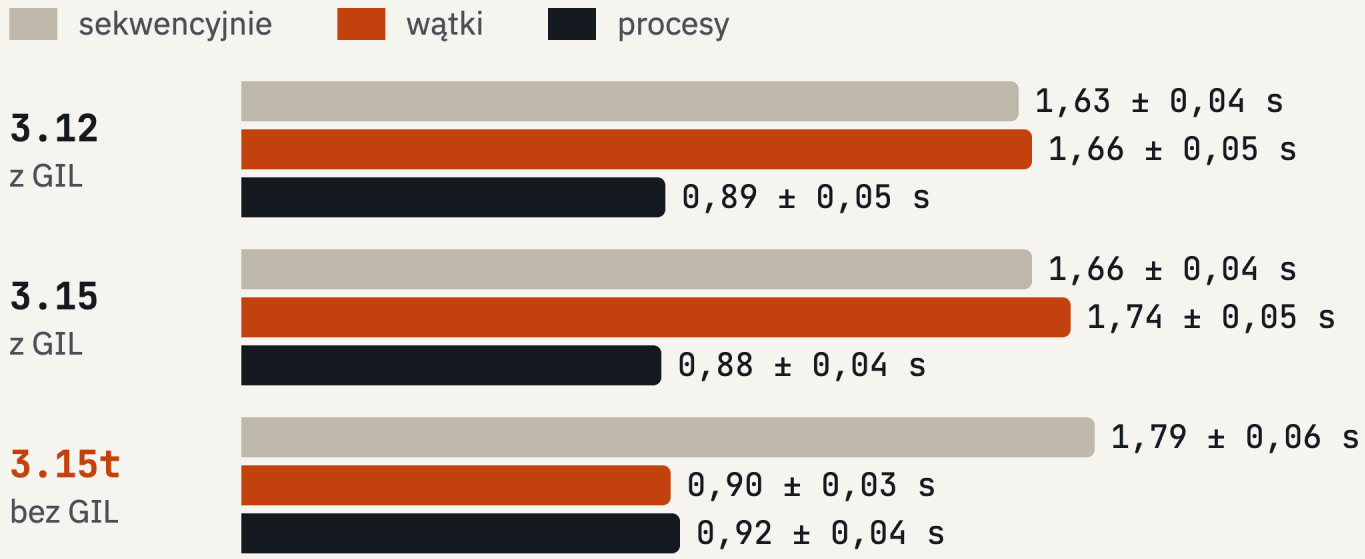
04 · BENCHMARK

Ta sama praca: sekwencyjnie, wątki, procesy

```python
def work(n):
    s = 0
    for i in range(n):
        s += i * i % 7
    return s
```

Docker, --cpuset-cpus=0-1 (2 rdzenie) · 40 mln iteracji · 10 powtórzeń: średnia ± odch. std. · z tworzeniem puli

Z GIL wątki ≈ 1×, procesy ≈ 2×. Bez GIL wątki też ≈ 2×.



Ten sam build dla każdej wersji (uv). Oficjalny obraz python:3.15-slim liczył ten sam kod ok. 40% wolniej — porównując różne obrazy, mierzysz też kompilator, nie tylko GIL.

Wątki i procesy jednym API

```
from concurrent.futures import ProcessPoolExecutor  # albo ThreadPoolExecutor
def partial(batch): ...                             # MAP: wynik dla jednej paczki
if __name__ == "__main__":                          # wymagane przy procesach
    batches = split(X, y, n=4)
    with ProcessPoolExecutor(max_workers=4) as ex:
        parts = list(ex.map(partial, batches))
    coef = combine(parts)                           # REDUCE: złożenie wyników
```

Start procesów

Linux: forkserver (od 3.14), macOS i Windows: spawn. Dlatego potrzebny if __name__.

Dane serializowane przez pickle

10 mln floatów to 90 MB i 0,5 s serializacji. Samo sum() na nich: 0,05 s.

Wątki bez GIL

Na 3.14t i 3.15t ten sam ThreadPoolExecutor liczy równolegle, bez zmian w kodzie.

06 · MAP-REDUCE

Regresja liniowa w paczkach

1 · MAP — każdy wątek lub proces liczy sumy dla swojej paczki

paczka 1 · x = 0…4

n=5 · Sx=10 · Sy=18 · Sxy=50 · Sxx=30

paczka 2 · x = 5…9

n=5 · Sx=35 · Sy=47 · Sxy=339 · Sxx=255

2 · REDUCE — dodaj sumy z paczek

n=10 · Sx=45 · Sy=65 · Sxy=389 · Sxx=285

3 · WSPÓŁCZYNNIKI — Sx = Σx, Sy = Σy, Sxy = Σx·y, Sxx = Σx²

```
b1 = (n*Sxy - Sx*Sy) / (n*Sxx - Sx**2)   # = 1.1697
b0 = (Sy - b1*Sx) / n                    # = 1.2364
```

07 · POMIAR

Jak rzetelnie mierzyć czas

```
import time, statistics
times = []
for _ in range(5):
    t0 = time.perf_counter()
    model.fit(X, y)
    times.append(time.perf_counter() - t0)
mean = statistics.mean(times)
std = statistics.stdev(times)
```

NumPy ma własne wątki (BLAS). Ustaw ich liczbę przed importem numpy, np. OPENBLAS_NUM_THREADS=1, i sprawdź przez threadpoolctl.

perf_counter(), nie time.time()
zegar monotoniczny o najwyższej rozdzielczości

Mierz samo fit()
bez wczytywania pliku i generowania danych

Powtórzenia i rozrzut
średnia ± odch. std. albo mediana; 1. przebieg to rozgrzewka

Zapisz środowisko
wersje Pythona i NumPy, liczba rdzeni i wątków BLAS

Wiele wersji Pythona w Dockerze

```
$ docker run --rm -v "$PWD":/app -w /app python:3.12-slim \
    bash -c "pip install -r requirements.txt && python -V"
```

requirements.txt z repo: czy jest gotowy wheel?

PAKIET	3.10	3.11	3.12	3.13	3.14
numpy 1.26.4	tak	tak	tak	**nie**	**nie**
pandas 1.5.3	tak	tak	**nie**	**nie**	**nie**

Sprawdzone przez pip download --only-binary=:all: dla Linuksa x86_64.

`-v "$PWD":/app`
kod i wyniki (data/) zostają na Twoim dysku

`nie = brak wheela`
pip buduje ze źródeł; obraz slim nie ma gcc

`--cpuset-cpus=0-3`
przypina rdzenie; przy --cpus Python dalej widzi wszystkie

09 · ZADANIA

Do zrobienia w domu

ZADANIE 4.1

Generator danych

Pary (x, y) z funkcji liniowej, np. y = 2x + 5, zapisane do <num-samples>.pkl.

`generate_data.py`

ZADANIE 4.2

Cztery regresje

Sekwencyjnie (bez NumPy), NumPy, wątki i procesy — dwie ostatnie na paczkach.

`lr/lr_*.py`

ZADANIE 4.3

Eksperymenty

Pomiary z powtórzeniami, wyniki w JSON, wykresy comparison.png i scaling.png.

`run_experiments.py`

ZADANIE 4.4

Wszystko naraz

Kilka zbiorów danych, eksperymenty i Docker z min. 3 wersjami Pythona i dobranymi zależnościami.

`run_all.sh`

Start: `pip install -e .` `pip install pytest` `pytest -s tests/`